

1. Institutional Context: Options & Volatility Risk on the Nomura CRB Desk

When interviewing with an Executive Director (ED) on the **Cash Equities Central Risk Book (CRB)** desk at Nomura, you must demonstrate a highly structured, rigorous understanding of risk-neutral replication. While a CRB desk primarily aggregates cash equity flows, its core operational mandate is managing, pricing, and systematically offloading residual inventory risk. Large blocks of institutional client orders often contain implied or embedded options, index derivatives, or structured equity swaps.

To manage this risk, the CRB desk uses dynamic delta-hedging, index options, or customized baskets to mitigate portfolio tracking errors. If you cannot explain how non-linear risk risks scale, how the option replication portfolio breaks even, or how the interaction between time decay (Θ) and curvature (Γ) impacts the firm's balance sheet, you cannot safely manage the book's capital.

Below is the first-principles mathematical derivation, structural breakdown, and production-ready simulation engine for derivative sensitivities and portfolio replication mechanics.

2. First-Principles Mathematical Derivation of the Greeks

The Black-Scholes Analytic Pricing Framework

The risk-neutral price of a European call option $C(S, \tau)$ with spot asset price S , strike price K , risk-free rate r , volatility σ , and time-to-maturity $\tau = T - t$ is given by:

$$C(S, \tau) = S\Phi(d_1) - Ke^{-r\tau}\Phi(d_2)$$

Where $\Phi(x)$ represents the standard normal cumulative distribution function (CDF):

$$\Phi(x) = \int_{-\infty}^x \phi(y) dy = \int_{-\infty}^x \frac{1}{\sqrt{2\pi}} e^{-\frac{y^2}{2}} dy$$

And $\phi(x)$ is the standard normal probability density function (PDF):

$$\phi(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}}$$

The standardized boundary variables d_1 and d_2 are defined as:

$$d_1 = \frac{\ln\left(\frac{S}{K}\right) + \left(r + \frac{\sigma^2}{2}\right)\tau}{\sigma\sqrt{\tau}}$$

$$d_2 = \frac{\ln\left(\frac{S}{K}\right) + \left(r - \frac{\sigma^2}{2}\right)\tau}{\sigma\sqrt{\tau}} = d_1 - \sigma\sqrt{\tau}$$

Derivation of the Core Algebraic Identity ($S\phi(d_1) = Ke^{-r\tau}\phi(d_2)$)

Before deriving the individual Greeks, we establish the fundamental algebraic identity that simplifies the calculus for Delta, Gamma, and Vega.

Step 1: Write out the explicit ratio of the standard normal densities evaluated at d_1 and d_2 :

$$\frac{\phi(d_1)}{\phi(d_2)} = \frac{\frac{1}{\sqrt{2\pi}} e^{-\frac{d_1^2}{2}}}{\frac{1}{\sqrt{2\pi}} e^{-\frac{d_2^2}{2}}} = e^{-\frac{1}{2}(d_1^2 - d_2^2)}$$

Step 2: Expand the difference of squares term $d_1^2 - d_2^2$ using the algebraic identity $a^2 - b^2 = (a - b)(a + b)$:

$$d_1^2 - d_2^2 = (d_1 - d_2)(d_1 + d_2)$$

Step 3: Substitute the structural relationship $d_1 - d_2 = \sigma\sqrt{\tau}$ into the expanded expression:

$$d_1^2 - d_2^2 = (\sigma\sqrt{\tau})(d_1 + d_2)$$

Step 4: Substitute the explicit definitions of d_1 and d_2 into the sum term $(d_1 + d_2)$:

$$d_1 + d_2 = \frac{\ln\left(\frac{S}{K}\right) + \left(r + \frac{\sigma^2}{2}\right)\tau}{\sigma\sqrt{\tau}} + \frac{\ln\left(\frac{S}{K}\right) + \left(r - \frac{\sigma^2}{2}\right)\tau}{\sigma\sqrt{\tau}}$$

$$d_1 + d_2 = \frac{2\ln\left(\frac{S}{K}\right) + 2r\tau}{\sigma\sqrt{\tau}}$$

Step 5: Multiply this sum by the term $(d_1 - d_2 = \sigma\sqrt{\tau})$:

$$d_1^2 - d_2^2 = (\sigma\sqrt{\tau}) \left[\frac{2\ln\left(\frac{S}{K}\right) + 2r\tau}{\sigma\sqrt{\tau}} \right] = 2\ln\left(\frac{S}{K}\right) + 2r\tau$$

Step 6: Substitute this result back into the exponential ratio derived in Step 1:

$$\frac{\phi(d_1)}{\phi(d_2)} = e^{-\frac{1}{2}(2\ln\left(\frac{S}{K}\right) + 2r\tau)} = e^{-\ln\left(\frac{S}{K}\right) - r\tau}$$

Step 7: Isolate the individual components using the properties of exponents ($e^{A+B} = e^A \cdot e^B$):

$$\frac{\phi(d_1)}{\phi(d_2)} = e^{\ln\left(\frac{K}{S}\right)} \cdot e^{-r\tau} = \frac{K}{S} e^{-r\tau}$$

Step 8: Cross-multiply to yield the final identity used in options pricing models:

$$\boxed{S\phi(d_1) = Ke^{-r\tau}\phi(d_2)}$$

Derivation of Delta (Δ)

Delta measures the first-order linear sensitivity of the option price to changes in the underlying asset price:
 $\Delta = \frac{\partial C}{\partial S}$.

Step 1: Apply the product rule for differentiation to the Black-Scholes call option formula with respect to S :

$$\Delta = \frac{\partial}{\partial S} [S\Phi(d_1)] - \frac{\partial}{\partial S} [Ke^{-r\tau}\Phi(d_2)]$$

$$\Delta = \left(1 \cdot \Phi(d_1) + S \cdot \frac{\partial \Phi(d_1)}{\partial S} \right) - K e^{-r\tau} \frac{\partial \Phi(d_2)}{\partial S}$$

Step 2: Apply the chain rule to differentiate the cumulative distribution functions $\Phi(d_1)$ and $\Phi(d_2)$:

$$\Delta = \Phi(d_1) + S \phi(d_1) \frac{\partial d_1}{\partial S} - K e^{-r\tau} \phi(d_2) \frac{\partial d_2}{\partial S}$$

Step 3: Differentiate the boundary variable equations d_1 and d_2 with respect to S :

$$\frac{\partial d_1}{\partial S} = \frac{\partial}{\partial S} \left[\frac{\ln S - \ln K + \left(r + \frac{\sigma^2}{2} \right) \tau}{\sigma \sqrt{\tau}} \right] = \frac{1}{S \sigma \sqrt{\tau}}$$

Since $d_2 = d_1 - \sigma \sqrt{\tau}$ and the second term is independent of S , their spatial partial derivatives are identical:

$$\frac{\partial d_2}{\partial S} = \frac{\partial d_1}{\partial S} = \frac{1}{S \sigma \sqrt{\tau}}$$

Step 4: Substitute these partial derivatives back into the expanded equation from Step 2:

$$\begin{aligned} \Delta &= \Phi(d_1) + S \phi(d_1) \left(\frac{1}{S \sigma \sqrt{\tau}} \right) - K e^{-r\tau} \phi(d_2) \left(\frac{1}{S \sigma \sqrt{\tau}} \right) \\ \Delta &= \Phi(d_1) + \frac{1}{S \sigma \sqrt{\tau}} [S \phi(d_1) - K e^{-r\tau} \phi(d_2)] \end{aligned}$$

Step 5: Apply our core algebraic identity ($S \phi(d_1) = K e^{-r\tau} \phi(d_2)$). The term inside the brackets reduces to zero, simplifying the expression to:

$$\Delta = \Phi(d_1) + \frac{1}{S \sigma \sqrt{\tau}} \cdot [0]$$

$$\boxed{\Delta = \Phi(d_1)}$$

Derivation of Gamma (Γ)

Gamma measures the second-order acceleration sensitivity of the option price to changes in the underlying asset price: $\Gamma = \frac{\partial^2 C}{\partial S^2} = \frac{\partial \Delta}{\partial S}$.

Step 1: Differentiate our derived expression for Delta ($\Delta = \Phi(d_1)$) with respect to S :

$$\Gamma = \frac{\partial}{\partial S} [\Phi(d_1)]$$

Step 2: Apply the chain rule:

$$\Gamma = \phi(d_1) \frac{\partial d_1}{\partial S}$$

Step 3: Substitute the spatial derivative $\frac{\partial d_1}{\partial S} = \frac{1}{S \sigma \sqrt{\tau}}$ derived during the Delta step:

$$\boxed{\Gamma = \frac{\phi(d_1)}{S \sigma \sqrt{\tau}}}$$

Derivation of Vega ($\mathcal{V}$)

Vega measures the first-order sensitivity of the option price to shifts in the asset's volatility parameter: $\mathcal{V} = \frac{\partial C}{\partial \sigma}$.

Step 1: Differentiate the Black-Scholes call framework with respect to σ :

$$\begin{aligned}\mathcal{V} &= \frac{\partial}{\partial \sigma} [S\Phi(d_1) - Ke^{-r\tau}\Phi(d_2)] \\ \mathcal{V} &= S\phi(d_1)\frac{\partial d_1}{\partial \sigma} - Ke^{-r\tau}\phi(d_2)\frac{\partial d_2}{\partial \sigma}\end{aligned}$$

Step 2: Compute the partial derivatives of d_1 and d_2 with respect to σ :

$$\begin{aligned}d_1 &= \frac{\ln\left(\frac{S}{K}\right) + r\tau}{\sigma\sqrt{\tau}} + \frac{1}{2}\sigma\sqrt{\tau} \\ \frac{\partial d_1}{\partial \sigma} &= -\frac{\ln\left(\frac{S}{K}\right) + r\tau}{\sigma^2\sqrt{\tau}} + \frac{1}{2}\sqrt{\tau}\end{aligned}$$

Now, compute the derivative for d_2 :

$$d_2 = d_1 - \sigma\sqrt{\tau} \implies \frac{\partial d_2}{\partial \sigma} = \frac{\partial d_1}{\partial \sigma} - \sqrt{\tau}$$

Step 3: Substitute these derivatives into the core expression from Step 1:

$$\mathcal{V} = S\phi(d_1)\frac{\partial d_1}{\partial \sigma} - Ke^{-r\tau}\phi(d_2)\left(\frac{\partial d_1}{\partial \sigma} - \sqrt{\tau}\right)$$

Step 4: Rearrange and factor the expressions to group the $\frac{\partial d_1}{\partial \sigma}$ terms together:

$$\mathcal{V} = [S\phi(d_1) - Ke^{-r\tau}\phi(d_2)]\frac{\partial d_1}{\partial \sigma} + Ke^{-r\tau}\phi(d_2)\sqrt{\tau}$$

Step 5: Apply our core algebraic identity. The bracketed term reduces to zero, simplifying the expression to:

$$\mathcal{V} = [0]\frac{\partial d_1}{\partial \sigma} + Ke^{-r\tau}\phi(d_2)\sqrt{\tau} = Ke^{-r\tau}\phi(d_2)\sqrt{\tau}$$

Step 6: Apply the identity one final time to substitute $S\phi(d_1)$ for $Ke^{-r\tau}\phi(d_2)$:

$$\boxed{\mathcal{V} = S\phi(d_1)\sqrt{\tau}}$$

3. Black-Scholes PDE Algebraic Rearrangement

The Black-Scholes Partial Differential Equation (PDE) governs any derivative asset whose value depends on a continuous, non-arbitrage underlying security:

$$\frac{\partial V}{\partial t} + rS\frac{\partial V}{\partial S} + \frac{1}{2}\sigma^2S^2\frac{\partial^2 V}{\partial S^2} = rV$$

Step-by-Step Mapping of Local Greek Expressions

To express the PDE in terms of portfolio Greeks, we map each partial derivative to its financial sensitivity metric:

1. The time derivative is defined as Theta (Θ): $\Theta = \frac{\partial V}{\partial t}$.
2. The first spatial derivative is defined as Delta (Δ): $\Delta = \frac{\partial V}{\partial S}$.
3. The second spatial derivative is defined as Gamma (Γ): $\Gamma = \frac{\partial^2 V}{\partial S^2}$.

Substitute these terms directly into the PDE:

$$\Theta + rS\Delta + \frac{1}{2}\sigma^2 S^2 \Gamma = rV$$

Step-by-Step Algebraic Factoring

Step 1: Subtract the term $rS\Delta$ from both sides of the equation:

$$\Theta + \frac{1}{2}\sigma^2 S^2 \Gamma = rV - rS\Delta$$

Step 2: Factor out the risk-free rate (r) from the right-hand side to establish the standard rearranged form:

$$\boxed{\Theta + \frac{1}{2}\sigma^2 S^2 \Gamma = r(V - S\Delta)}$$

4. P&L Mechanics & Economic Intuition (Gamma Scalping vs. Theta Decay)

The Replicating Portfolio Framework

Consider a long position in a European derivative V that is continuously delta-hedged. We construct a self-financing portfolio Π :

$$\Pi = V - \Delta S$$

The cash required to establish this position is $V - \Delta S$. If this cash balance is positive, it earns the risk-free rate; if negative, it borrows at r . This funding cost is represented by the right side of our rearranged PDE: $r(V - S\Delta)$.

Itô's Lemma & Realized Volatility Slippage

Suppose we maintain a zero-delta position (Π) over an incremental time step dt . The asset's true price process evolves according to a geometric Brownian motion governed by a localized *realized* volatility parameter σ_{realized} :

$$dS_t = \mu S_t dt + \sigma_{\text{realized}} S_t dW_t$$

By applying Itô's Lemma, the empirical physical change in the value of our option position (dV) is given by:

$$dV = \left(\frac{\partial V}{\partial t} + \mu S \frac{\partial V}{\partial S} + \frac{1}{2} \sigma_{\text{realized}}^2 S^2 \frac{\partial^2 V}{\partial S^2} \right) dt + \sigma_{\text{realized}} S \frac{\partial V}{\partial S} dW_t$$

Substitute our Greek definitions (Θ, Δ, Γ) into the expression:

$$dV = \left(\Theta + \mu S \Delta + \frac{1}{2} \sigma_{\text{realized}}^2 S^2 \Gamma \right) dt + \sigma_{\text{realized}} S \Delta dW_t$$

The total physical change in the value of our hedged portfolio ($d\Pi = dV - \Delta dS$) is:

$$d\Pi = \left(\Theta + \mu S \Delta + \frac{1}{2} \sigma_{\text{realized}}^2 S^2 \Gamma \right) dt + \sigma_{\text{realized}} S \Delta dW_t - \Delta (\mu S dt + \sigma_{\text{realized}} S dW_t)$$

The directional drift (μ) and stochastic noise (dW_t) terms cancel out, leaving:

$$d\Pi = \left(\Theta + \frac{1}{2} \sigma_{\text{realized}}^2 S^2 \Gamma \right) dt$$

Derivation of the Strategic Break-Even Equation

To find the portfolio's net excess return (or replication slippage), we subtract the financing cost of the cash account from the portfolio's physical return:

$$\text{Net P\&L} = d\Pi - r(V - \Delta S)dt$$

$$\text{Net P\&L} = \left(\Theta + \frac{1}{2} \sigma_{\text{realized}}^2 S^2 \Gamma \right) dt - r(V - \Delta S)dt$$

From our rearranged Black-Scholes PDE, the cost of carrying the cash account can be expressed as:

$$r(V - S\Delta) = \Theta + \frac{1}{2} \sigma_{\text{implied}}^2 S^2 \Gamma$$

Substitute this back into the Net P&L equation:

$$\text{Net P\&L} = \left(\Theta + \frac{1}{2} \sigma_{\text{realized}}^2 S^2 \Gamma \right) dt - \left(\Theta + \frac{1}{2} \sigma_{\text{implied}}^2 S^2 \Gamma \right) dt$$

The time-decay terms (Θ) cancel out, yielding the fundamental equation for volatility trading:

$$\text{Net P\&L} = \frac{1}{2} S^2 \Gamma (\sigma_{\text{realized}}^2 - \sigma_{\text{implied}}^2) dt$$

Operational Interpretation for the Nomura CRB Desk

This relationship highlights the core mechanics of managing non-linear risk on a trading desk:

1. **The Theta-Gamma Trade-off:** For a long option position, Gamma is positive ($\Gamma > 0$) and Theta is negative ($\Theta < 0$). The position loses money over time due to time decay (Θdt). To offset this decay, the underlying asset must move enough to generate trading profits from convexity ($\frac{1}{2} \sigma^2 S^2 \Gamma dt$), a process known as **Gamma Scalping**.
2. **The Break-Even Condition:** The portfolio breaks even if and only if the realized volatility exactly matches the implied volatility embedded in the option's entry premium ($\sigma_{\text{realized}} = \sigma_{\text{implied}}$).
3. **Inventory Management Risk:** If the CRB desk internalizes a large customer block trade and takes on short options exposure ($\Gamma < 0$), it collects premium over time ($\Theta > 0$). However, it faces significant risk if the market experiences large, sudden moves. If realized market volatility exceeds the implied entry threshold ($\sigma_{\text{realized}} > \sigma_{\text{implied}}$), the desk will incur replication losses, which must be actively managed.

5. Production-Grade Simulation Engine & Interactive Visualization Panel

This production-ready Python simulation engine models a discrete delta-hedging strategy under mismatched volatility regimes. It calculates exact Black-Scholes prices and Greeks from first principles, maintains a self-financing cash account, tracks transaction costs, and generates an interactive risk management dashboard using Plotly.

```
import numpy as np
import pandas as pd
import scipy.stats as stats
import plotly.graph_objects as go
from plotly.subplots import make_subplots

class CRBVolatilityEngine:
    """
    Production-grade simulation framework for delta-hedging and risk replication.
    Calculates analytical options metrics and tracks cash accounting from first
    principles.
    """
    def __init__(self, S0: float, K: float, r: float, sigma_implied: float, T:
float, steps: int):
        self.S0 = S0
        self.K = K
        self.r = r
        self.sigma_implied = sigma_implied
        self.T = T
        self.steps = steps
        self.dt = T / steps

    def calculate_bs_greeks(self, S: float, tau: float) -> dict:
        """
        Computes the option price, Delta, Gamma, Vega, and Theta from first
        principles.
        """
        # Handle expiration edge case safely
        if tau <= 1e-6:
            return {
                "price": max(S - self.K, 0.0), "delta": 1.0 if S > self.K else
0.0,
                "gamma": 0.0, "vega": 0.0, "theta": 0.0
            }

        d1 = (np.log(S / self.K) + (self.r + 0.5 * self.sigma_implied**2) * tau) /
(self.sigma_implied * np.sqrt(tau))
        d2 = d1 - self.sigma_implied * np.sqrt(tau)

        price = S * stats.norm.cdf(d1) - self.K * np.exp(-self.r * tau) *
stats.norm.cdf(d2)
        delta = stats.norm.cdf(d1)

        phi_d1 = stats.norm.pdf(d1)
```

```

gamma = phi_d1 / (S * self.sigma_implied * np.sqrt(tau))
vega = S * phi_d1 * np.sqrt(tau)

# Call Option Theta derivation
theta = -(S * phi_d1 * self.sigma_implied) / (2.0 * np.sqrt(tau)) - \
        self.r * self.K * np.exp(-self.r * tau) * stats.norm.cdf(d2)

return {"price": price, "delta": delta, "gamma": gamma, "vega": vega,
        "theta": theta}

def simulate_replication_path(self, sigma_realized: float, fee_bps: float =
1.0) -> pd.DataFrame:
    """
    Simulates asset price paths and tracks the performance of a self-financing
    delta-hedged replicating portfolio.
    """
    np.random.seed(13458) # Set seed to the job code from NOMURA_JD.md

    # Pre-allocate tracking arrays
    prices = np.zeros(self.steps + 1)
    times = np.zeros(self.steps + 1)
    option_vals = np.zeros(self.steps + 1)
    deltas = np.zeros(self.steps + 1)
    gammas = np.zeros(self.steps + 1)
    thetas = np.zeros(self.steps + 1)
    cash_account = np.zeros(self.steps + 1)
    portfolio_value = np.zeros(self.steps + 1)
    pnl_slippage = np.zeros(self.steps + 1)

    prices[0] = self.S0
    greeks_0 = self.calculate_bs_greeks(self.S0, self.T)

    option_vals[0] = greeks_0["price"]
    deltas[0] = greeks_0["delta"]
    gammas[0] = greeks_0["gamma"]
    thetas[0] = greeks_0["theta"]

    # Replicating portfolio initialization: Buy the option, short the delta
    stock leg
    # Cash account balance = Value of stock shorted - Cost of option premium
    cash_account[0] = deltas[0] * self.S0 - option_vals[0]
    portfolio_value[0] = option_vals[0] - deltas[0] * self.S0 +
cash_account[0]

    for t in range(1, self.steps + 1):
        tau_current = self.T - t * self.dt
        times[t] = t * self.dt

        # Evolve asset price via Geometric Brownian Motion using realized
volatility
        z = np.random.normal(0, 1)
        prices[t] = prices[t-1] * np.exp((self.r - 0.5 * sigma_realized**2) *
self.dt +
sigma_realized * np.sqrt(self.dt) *

```



```

z)

    # Compute updated option price and Greeks
    greeks = self.calculate_bs_greeks(prices[t], tau_current)
    option_vals[t] = greeks["price"]
    deltas[t] = greeks["delta"]
    gammas[t] = greeks["gamma"]
    thetas[t] = greeks["theta"]

    # Accrue interest on the cash account over the time step dt
    cash_accrued = cash_account[t-1] * np.exp(self.r * self.dt)

    # Determine adjustments needed for the delta hedge
    shares_to_trade = deltas[t] - deltas[t-1]
    trade_cost = shares_to_trade * prices[t]
    transaction_fee = abs(trade_cost) * (fee_bps / 10000.0)

    # Update cash account balance after execution and trading fees
    cash_account[t] = cash_accrued - trade_cost - transaction_fee

    # Net portfolio valuation: Option + Stock Position + Cash Account
    portfolio_value[t] = option_vals[t] - deltas[t] * prices[t] +
cash_account[t]

    # Track theoretical vs realized replication slippage
    theoretical_gamma_profit = 0.5 * gammas[t-1] * (prices[t] - prices[t-
1])**2
    theoretical_theta_loss = thetas[t-1] * self.dt
    pnl_slippage[t] = pnl_slippage[t-1] + (theoretical_gamma_profit +
theoretical_theta_loss)

    return pd.DataFrame({
        "Time": times, "Price": prices, "OptionValue": option_vals,
        "Delta": deltas, "Gamma": gammas, "Theta": thetas,
        "Cash": cash_account, "PortfolioValue": portfolio_value, "Slippage":
pnl_slippage
    })

    def generate_dashboard(self, df_high_vol: pd.DataFrame, df_low_vol:
pd.DataFrame) -> go.Figure:
        """
        Generates a comprehensive risk dashboard analyzing replication tracking
errors,
        Greek profiles, and hedging performance across different volatility
regimes.
        """
        fig = make_subplots(
            rows=2, cols=2,
            subplot_titles=(
                "Underlying Asset Trajectory Profiles",
                "Total Replicating Portfolio Net Profit/Loss ($)",
                "Dynamic Curvature Evolution (Gamma Spatial Spectrum)",
                "Theta-Gamma Complementary Trade-Off Curve"
            ),

```

```

        vertical_spacing=0.15, horizontal_spacing=0.10
    )

    # Panel 1: Underlying Price Path
    fig.add_trace(
        go.Scatter(x=df_high_vol["Time"], y=df_high_vol["Price"],
mode="lines",
                    name="High Vol Path (Realized > Implied)",
line=dict(color="#1f77b4", width=2.5)),
        row=1, col=1
    )
    fig.add_trace(
        go.Scatter(x=df_low_vol["Time"], y=df_low_vol["Price"], mode="lines",
                    name="Low Vol Path (Realized < Implied)",
line=dict(color="#aec7e8", width=2)),
        row=1, col=1
    )
    fig.update_yaxes(title_text="Asset Price ($)", row=1, col=1)

    # Panel 2: Net Portfolio Return
    fig.add_trace(
        go.Scatter(x=df_high_vol["Time"], y=df_high_vol["PortfolioValue"],
mode="lines",
                    name="Net P&L: Realized Vol > Implied Vol",
line=dict(color="#2ca02c", width=2.5)),
        row=1, col=2
    )
    fig.add_trace(
        go.Scatter(x=df_low_vol["Time"], y=df_low_vol["PortfolioValue"],
mode="lines",
                    name="Net P&L: Realized Vol < Implied Vol",
line=dict(color="#d62728", width=2.5)),
        row=1, col=2
    )
    fig.update_yaxes(title_text="Portfolio Value Delta ($)", row=1, col=2)

    # Panel 3: Gamma Spectrum
    fig.add_trace(
        go.Scatter(x=df_high_vol["Price"], y=df_high_vol["Gamma"],
mode="markers",
                    name="Gamma Trace", marker=dict(color="#ff7f0e", size=4,
opacity=0.8)),
        row=2, col=1
    )
    fig.update_xaxes(title_text="Underlying Spot Price ($)", row=2, col=1)
    fig.update_yaxes(title_text="Gamma Value", row=2, col=1)

    # Panel 4: Theta-Gamma Trade-off
    fig.add_trace(
        go.Scatter(x=df_high_vol["Time"], y=df_high_vol["Theta"],
mode="lines",
                    name="Theta Decay Path", line=dict(color="#9467bd",
width=2)),
        row=2, col=2
    )

```

```

    )
    fig.update_yaxes(title_text="Theta Decay Density", row=2, col=2)

    # Global Layout Adjustments
    fig.update_layout(
        title=dict(
            text="<b>Nomura Securities Cash Equities CRB Quantitative
Analytics Panel</b><br>"
            "<sup>Continuous Delta Replicating Systems & First-Principles
Non-Linear Greek Testing</sup>",
            x=0.05, font=dict(size=16, family="Arial")
        ),
        template="plotly_white", height=780, width=1200, showlegend=True
    )

    for r in [1, 2]:
        for c in [1, 2]:
            if not (r == 2 and c == 1): # Skip custom x-axis label for panel
3
                fig.update_xaxes(title_text="Time Horizon Vector (Years)",
row=r, col=c)

    return fig

if __name__ == "__main__":
    # Initialize parameters matching an institutional option book profile
    initial_spot = 100.0
    strike_target = 100.0
    risk_free = 0.04 # 4.0% interest rate environment
    implied_vol_surface = 0.20 # 20.0% Implied Volatility model pricing baseline
    expiry_horizon = 0.25 # 3-month option duration window
    simulation_steps = 252 # Daily rebalancing frequency over the horizon

    engine = CRBVolatilityEngine(
        S0=initial_spot, K=strike_target, r=risk_free,
        sigma_implied=implied_vol_surface, T=expiry_horizon,
steps=simulation_steps
    )

    # Run simulation scenarios across different volatility regimes
    print("="*75)
    print("      NOMURA GLOBAL MARKETS OPTION REPLICATION SYSTEM RUNNING
")
    print("="*75)

    # Scenario A: Realized Volatility higher than Implied (28% vs 20%) -> Net
positive long gamma replication
    df_high = engine.simulate_replication_path(sigma_realized=0.28, fee_bps=0.0)
    # Scenario B: Realized Volatility lower than Implied (12% vs 20%) -> Net
negative long gamma replication
    df_low = engine.simulate_replication_path(sigma_realized=0.12, fee_bps=0.0)

    print(f"Target Implied Volatility Entry Baseline : {implied_vol_surface *
100:.1f}%")

```

```
print("-"*75)
print(f"Scenario 1 Net Realized P&L (High Vol 28%) :
${df_high['PortfolioValue'].iloc[-1]:.2f}")
print(f"Scenario 2 Net Realized P&L (Low Vol 12%) :
${df_low['PortfolioValue'].iloc[-1]:.2f}")
print("-"*75)
print("System execution completed. Metrics generated cleanly.")
print("-"*75)

# Display the interactive analytical panel
# dashboard = engine.generate_dashboard(df_high, df_low)
# dashboard.show()
```